DIWA Thoughts & Tasks Graph Explorer

Document type: Product + technical design recommendation **Status:** Documentation/design only — implementation intentionally deferred **Repository:** `emperorKDSR/obsidian_diwa` **Version context:** `7.3.48` (`package.json`) **Audience:** Product/design council, engineering council, Obsidian architecture review, Life-OS planning

Executive summary

This recommendation proposes a **desktop-first, dedicated graph exploration window** for DIWA thoughts and Gawa tasks. The feature should open from a selected thought or task in **DIWA or Gawa** into a **new Obsidian popout workspace window** centered on that seed item. The graph is not meant to be a novelty visualization. It is meant to make DIWA's already-existing relationship data useful for **exploration, enrichment, synthesis, retrospectives, orphan triage, and action**.

The recommended product direction is:

1. **Ship a dedicated Graph Explorer window, not an embedded panel.**
2. **Use a new registered Obsidian** `ItemView` **opened with** `workspace.getLeaf('window')`**, not** `window.open`**.**
3. **Build from existing** `IndexService` **indices and current entity relationships.**
4. **Keep the node/edge vocabulary minimal and high-signal.**
5. **Make the inspector action-oriented so every exploration session can lead to work.**
6. **Roll out in phases, beginning with a bounded, seeded graph and strong lifecycle/performance guardrails.**

This document is intentionally a **plan/design artifact only**. It does **not** authorize or perform implementation.

Recommended product direction

The product call

DIWA should eventually add a feature named **DIWA Thoughts & Tasks Graph Explorer**.

The recommended experience is a **new-window graph workspace** that opens around a selected thought or task from DIWA or Gawa, then lets the user:

- inspect connected thoughts, tasks, projects, and contexts
- expand the local graph safely
- enrich the selected node or selected cluster
- convert exploration into action through contextual commands

What the feature is for

The graph should primarily support:

- **unfinished thinking**
- **pre-synthesis clustering**
- **project memory and genealogy**
- **orphan thought triage**
- **task reflection loop completion**
- **retrospective analysis and enrichment**

What the feature is not for

The graph should **not** be positioned as:

- a vault-wide gimmick map
- a passive visualization layer
- a replacement for DIWA, Gawa, Synthesis, or Projects
- an excuse to render every possible relationship in the vault

The graph should be a **power workspace for structured thinking and action**, especially on desktop.

Why this is worth building later

The repository already contains most of the data needed for a useful graph:

- `IndexService` already maintains `thoughtIndex`, `taskIndex`, `projectIndex`, and related derived state.
- `ThoughtEntry` and `TaskEntry` already expose fields such as `links`, `context`, `project`, `sourceThoughtIds`, `reflectionThoughtId`, `wikilinks`, `topic`, `pinned`, and synthesis state.
- Existing desktop views already establish patterns for:
- dedicated `ItemView` workspaces
- desktop popout windows
- persisted lightweight view state
- resizable right-side inspection panels
- keyboard-oriented desktop interaction

The graph therefore does not require inventing a new data universe. It mainly requires a future implementation that can **compile and present the invisible structure that already exists**.

Council synthesis

UI/UX council perspective

The design council position is that the graph must feel like a **focused analysis workspace**, not a decorative diagram.

Core design takeaways:

- open around a **seed thought or task**, never as an unbounded full-vault default
- keep the interface **minimalist, modern, and action-oriented**
- provide clear entry points from **DIWA** and **Gawa**
- use a layout optimized for **large-screen productivity**
- ensure strong empty states, filters, controls, and side inspection
- keep the graph legible under normal desktop usage without second-guessing the user

Engineering council perspective

The engineering council position is that this feature is viable **only if it is bounded and uses existing indices as source of truth**.

Core engineering takeaways:

- use the existing indexed entities instead of new ad hoc scans
- separate graph compilation from view rendering
- define canonical IDs carefully to prevent duplicate-node bugs
- constrain graph size with node/edge caps and seeded depth limits
- phase the rollout rather than shipping a maximal graph on day one

Obsidian architecture council perspective

The Obsidian architecture council position is firm:

- use a **new registered** `ItemView`
- open it through `workspace.getLeaf('window')`
- persist only **lightweight view state**
- use `ownerDocument` **/** `defaultView` for popout-safe DOM and listeners
- wire refresh and cleanup through existing Obsidian/plugin lifecycle patterns

Life-OS council perspective

The Life-OS position is that the graph matters only if it helps the user **finish thinking, synthesize, reflect, and act**.

Core Life-OS takeaways:

- the graph should surface high-value cognitive structures already present in DIWA
- the best use cases are not passive browsing but **orphan triage, synthesis preparation, project retrospectives, and reflection loop completion**
- node/edge vocabulary must stay minimal and meaningful
- every graph session should offer next actions such as:
- convert to task
- synthesize selected
- analyze with AI
- link items
- open source note/task
- create or complete reflection

Final synthesis across councils

All councils converge on the same conclusion:

> The right product is a **dedicated, seeded, desktop-first Graph Explorer window** that turns existing DIWA thought/task relationships into a useful exploration-and-enrichment workspace, while staying bounded, Obsidian-native, and action-oriented.

Explicit recommended decisions

Decision	Recommendation
Delivery now	**Do not implement now**; treat this as an approved recommendation only
Window model	Dedicated graph popout window
Obsidian integration	New registered `ItemView`
New-window opening	`workspace.getLeaf('window')`
Entry model	Open from DIWA/Gawa on a selected seed thought/task
Default scope	Seed-bounded graph, not vault-wide
Source of truth	`IndexService` and current indices
Persisted state	Seed, filters, layout mode, selection, inspector state

Decision	Recommendation
Non-persisted state	Rendered graph state, physics/layout internals, DOM state
Node vocabulary (Phase 1)	Thought, Task, Project, Context
Edge vocabulary (Phase 1)	Explicit thought links, task-thought lineage, reflection links, project membership, context membership
Graph posture	Exploration + enrichment + action
Rollout	Phased

UX proposal

Entry points

The graph should be reachable from both DIWA and Gawa with clear, contextual entry points.

DIWA entry points

Recommended entry points:

- thought row/action menu in desktop DIWA surfaces
- thought inspection surfaces where a single thought is already in focus
- future command palette action such as **“Open Graph Explorer”** with a picker when no seed is preselected

Gawa entry points

Recommended entry points:

- task row/action menu in Gawa
- task detail/inspection affordance in task-centric views
- project views when the user wants to inspect the thought-to-task chain around a project

Opening behavior

Clicking **Graph** from DIWA or Gawa should:

1. resolve the selected seed thought/task
2. open a new dedicated workspace popout window
3. set the graph view state to that seed

4. render the local connection graph for exploration

The intended emotional feel is: **“open a second-monitor analysis cockpit for this item.”**

Desktop/new-window behavior

Desktop-first stance

This should be a **desktop-first experience**. The graph’s best form factor is a persistent analysis window with enough space for:

- a dominant canvas
- multi-pane inspection
- keyboard shortcuts
- resizing
- sustained exploration

Mobile and tablet support should be deferred.

Window behavior recommendation

- open as an Obsidian workspace popout leaf
- reuse an existing graph popout when one already exists unless the future product explicitly chooses multi-graph windows
- let Obsidian own popout window bounds via workspace layout persistence
- let the graph view persist only its own lightweight view state

Recommended layout

The recommended layout is:

- **Topbar**
- **Left rail**
- **Dominant center canvas**
- **Right inspector**
- **Bottom status bar**

Topbar

Should contain:

- graph title
- current seed breadcrumb
- back/forward seed history
- layout selector
- depth/scope selector
- filter access
- fit/recenter actions

Left rail

Should contain:

- node type toggles
- graph scope filters
- context/project/status filters
- legend
- zoom controls

Center canvas

Should contain:

- graph visualization
- hover/selection states
- bounded expansion controls
- mini-map only if it improves usability under real node counts

Right inspector

Should contain:

- selected node summary
- metadata
- linked items list
- primary enrichment actions
- multi-select bulk actions when applicable

The inspector should be **resizable**, following existing desktop pane ergonomics already present in the codebase.

Bottom status bar

Should contain:

- node count
- edge count
- active filter summary
- truncation notice when caps are hit
- lightweight status/refresh hints

Interaction model

Core interaction rules

- **Single click:** select node and load inspector
- **Double click / focus action:** make that node the new seed
- **Shift-click / multi-select:** create an actionable cluster selection
- **Canvas click:** clear selection and return inspector to seed context
- **Expand neighbors:** user-controlled, local, bounded
- **Hide/filter:** non-destructive; changes visibility, not data

Keyboard model

Recommended keyboard support:

- arrow/tab navigation between focusable nodes
- enter/select current node
- escape/clear selection
- fit graph to view
- back to previous seed
- inspector resize shortcuts on desktop

Empty states

The graph must have explicit empty states.

Empty state types

1. **No connected nodes under current filters**

- explain why the graph is sparse
- offer actions to widen filters or increase depth

1. **True orphan item**

- explain that no meaningful links exist yet
- suggest enrichment actions such as linking, adding context, or converting to task

1. **Truncated graph**

- explain that node/edge caps limited the render
- suggest refining filters or focusing on a smaller seed scope

Loading and refresh states

The graph should never appear as a blank void.

Recommended behavior:

- render shell immediately
- render seed first
- progressively resolve and display neighbors
- show a clear “building graph” status when recomputing

Action and enrichment workflows

This feature should be designed around turning exploration into progress.

Primary enrichment actions

Recommended primary actions by context:

Context	Recommended actions
Selected thought	Edit, link, convert to task, assign/refine context, analyze with AI, open source note
Selected task	Edit, inspect origin thought(s), inspect reflection, open source task, analyze with AI
Selected cluster of thoughts	Synthesize selected, link selected, analyze cluster, assign context/project

Context	Recommended actions
Selected project-centered subgraph	Analyze project memory, inspect genealogy, identify missing reflection loops
Orphan thought	Link, assign context, convert to task, archive intentionally

Highest-value workflows

1. Pre-synthesis clustering

Open a raw or unsynthesized thought from DIWA, reveal nearby related thoughts/tasks/projects, select a cluster, then trigger **Synthesize Selected**.

2. Project memory / genealogy

Open from a project-related thought or task, inspect the full chain of:

- related thoughts
- spawned tasks
- associated project items
- post-task reflections

This supports retrospectives and project memory.

3. Orphan thought triage

Open a thought with weak structure, see that it is effectively disconnected, and then resolve that disconnection through linking, contextualization, synthesis, or conversion to action.

4. Reflection loop completion

Open a task from Gawa and inspect:

- source thought(s)
- downstream reflection thought
- sibling project work

This closes the loop between thinking, doing, and learning.

Recommended data model

Phase 1 node model

Phase 1 should keep nodes minimal:

- **Thought**
- **Task**
- **Project**
- **Context**

Why this vocabulary

This is the best balance of:

- usefulness
- readability
- implementation realism
- Life-OS signal quality

Explicit Phase 1 non-nodes

Do **not** make these first-class graph nodes in Phase 1:

- dues
- habits
- finance entities
- arbitrary vault files
- rendered dates as a default graph structure
- speculative AI-only entities

Phase 1 edge model

Recommended Phase 1 edges:

- **thought** ↔ **thought** via explicit thought links
- **thought** → **task** where task lineage exists
- **task** → **thought** for reflection and source-thought relationships
- **thought/task** → **project**
- **thought/task** → **context**

Deferred edge model

Possible later additions, but not Phase 1 defaults:

- resolved wikilink edges where the target is a known DIWA entity
- topic cluster edges
- AI-suggested relationship overlays

Those should be opt-in and only added after the basic graph proves useful.

Canonical identity strategy

This is a critical correctness requirement.

The implementation should eventually use **typed canonical graph IDs** so different entity types cannot collide.

Recommended strategy:

- thought node ID: `thought::<filePath>`
- task node ID: `task::<taskId || filePath>`
- project node ID: `project::<projectId>`
- context node ID: `context::<normalizedContext>`

Why this matters

The current repository mixes stable IDs and file-path-based identity across entity types. Without a canonical typed ID strategy, the eventual feature risks:

- duplicate nodes
- broken selection state
- incorrect edge wiring
- inconsistent refresh behavior after renames or legacy-data resolution

This decision should be treated as **non-negotiable architecture** for the future implementation.

Graph generation strategy

Source of truth

Use `IndexService` and existing in-memory indices as the source of truth.

The future graph compiler should be a read-only layer that consumes:

- `thoughtIndex`
- `taskIndex`
- `projectIndex`
- existing relationship fields on indexed entities

It should **not** re-scan the vault from the view layer.

Recommended graph compilation approach

1. resolve the seed entity
2. build or reuse adjacency based on indexed entities
3. traverse outward up to a bounded depth
4. compile visible nodes and edges after filters are applied
5. enforce caps before rendering
6. return a pure graph snapshot to the view

Default scoping recommendation

Recommended defaults:

- seed-bounded
- default depth: **2 hops**
- user-expandable depth for local exploration
- conservative filters on first open

This avoids hairball graphs and keeps the first render useful.

Architecture recommendation

Future component boundaries

When this is eventually built, the cleanest architecture is:

1. Graph Explorer View

A new registered `ItemView` responsible for:

- lifecycle
- rendering shell
- canvas/SVG interaction
- inspector UI
- lightweight state persistence
- graph refresh orchestration

2. Graph Service / compiler layer

A pure, read-only layer responsible for:

- converting indexed DIWA entities into graph nodes/edges
- canonical ID normalization
- scoping and filtering
- cap enforcement
- returning a renderable snapshot

3. Existing services remain source systems

The graph should call back into existing services/controllers/modals for actions such as:

- edit
- link
- convert to task
- synthesize
- AI analysis
- open file/note/task

The graph should not become a new source system.

Obsidian-native implementation guidance

Required view/window approach

When implementation eventually begins, the feature should:

- register a new `ItemView`

- open it as a workspace popout leaf via `workspace.getLeaf('window')`
- use `getState()` / `setState()` for lightweight restoration

Persist only lightweight view state

Persist:

- seed ID
- filters
- layout mode
- selection state
- inspector/open panel state if small enough

Do **not** persist:

- rendered node positions
- simulation state
- cached snapshots that can be rebuilt
- DOM state
- canvas pixels

Popout-safety requirements

The eventual implementation must be popout-safe.

That means:

- derive DOM APIs from the leaf's `ownerDocument`
- derive timers/observers/window interactions from `defaultView`
- never assume global `window` / `document` are the correct host for the popout

Refresh and cleanup expectations

The eventual implementation should:

- integrate with current plugin refresh wiring rather than inventing its own indexing path
- debounce rebuilds
- clean up all listeners, timers, observers, and rendering resources in `onClose()`
- avoid synchronous heavy rebuild work inside raw event callbacks

Performance and scoping guardrails

Recommended caps

Recommended initial limits:

- **initial render target:** ~120 nodes / ~240 edges
- **hard cap:** ~200 nodes / ~400 edges
- **per-node expansion cap:** ~12–20 neighbors

These values can be refined later, but a bounded Phase 1 is essential.

Recommended performance rules

- never default to full-vault graphing
- debounce full rebuilds on index refresh
- skip unnecessary rebuilds when the visible entity types are unaffected
- show explicit truncation messaging when caps are reached
- optimize for clarity before density

Why bounded scope matters

A graph that is technically exhaustive but visually useless fails the feature goal. The product should prefer:

- fast open
- trustworthy graph shape
- clear inspector actions
- meaningful local exploration

over maximal relationship density.

Implementation phases for later execution

Phase 1 — Foundation

Goal: deliver a correct, bounded, desktop-first graph workspace.

Should include:

- registered graph `ItemView`
- new-window opening from DIWA/Gawa seed items
- seeded graph compilation from existing indices
- Phase 1 node/edge vocabulary
- filters and inspector
- keyboard basics
- refresh wiring
- popout-safe lifecycle handling
- node/edge caps

Should not include yet:

- mobile/tablet graph support
- expansive AI graph features
- full-vault default rendering
- broad non-DIWA vault graphing

Phase 2 — Enrichment

Possible additions:

- resolved wikilink overlays
- improved layout modes
- better project/subgraph workflows
- export/sharing of graph snapshots inside DIWA workflows

Phase 3 — Advanced intelligence

Only if the bounded graph proves valuable:

- AI-assisted cluster labeling
- AI-suggested relatedness overlays
- richer retrospective workflows

Any AI layer should remain **assistive, not authoritative**.

Risks and tradeoffs

1. Dedicated window vs embedded panel

Debate

- embedded panel would be quicker and reuse existing layouts
- dedicated window better matches the desired desktop analysis experience

Recommendation

Use a **dedicated new window**. The graph should feel like a second workspace, not a cramped side panel.

2. Seed-bounded graph vs full-vault graph

Debate

- full-vault graph is familiar and comprehensive
- seeded graph is more useful, faster, and more legible

Recommendation

Use a **seed-bounded graph by default**.

3. External graph library vs minimal custom rendering

Debate

- external library may accelerate implementation and layout quality
- custom rendering reduces bundle weight and dependency overhead

Recommendation

Start with the smallest practical rendering approach that satisfies the bounded Phase 1 scope. Revisit dependency decisions only if actual graph complexity demands it.

4. Broad node vocabulary vs minimal high-signal vocabulary

Debate

- more node types can make the graph look richer

- too many types reduce signal and create cognitive clutter

Recommendation

Keep Phase 1 vocabulary minimal: **Thought, Task, Project, Context**.

5. Persisting more graph state vs only lightweight state

Debate

- persisting layout/simulation state may feel convenient
- persisting too much state increases fragility and violates Obsidian lifecycle expectations

Recommendation

Persist only **lightweight view state**.

Non-goals

This recommendation intentionally does **not** propose immediate execution of:

- implementation in TypeScript
- new view registration in code
- new commands/ribbon actions in code
- graph rendering engine integration
- data model migration
- tests

This is a **planning/design deliverable only**.

Final recommendation

DIWA should eventually build the **Thoughts & Tasks Graph Explorer** as a **desktop-first, Obsidian-native, dedicated popout workspace** centered on a seed thought or task from DIWA or Gawa.

The feature should be judged against one standard:

> Does it help the user move from connected information to better action, synthesis, reflection, and project understanding?

If yes, it is aligned with DIWA. If it becomes merely a pretty graph, it is not.

The strongest version of the feature is therefore:

- **seeded**
- **bounded**
- **desktop-first**
- **action-oriented**
- **built on current indices**
- **safe within Obsidian lifecycle patterns**
- **rolled out in phases**

Implementation should remain deferred until explicitly authorized.

Repository references used for this recommendation

- `package.json`
- `src/main.ts`
- `src/constants.ts`
- `src/types.ts`
- `src/services/IndexService.ts`
- `src/application/RefreshCoordinator.ts`
- `src/views/DesktopHubView.ts`
- `src/views/SearchView.ts`
- `src/views/ThoughtFocusPanel.ts`
- `src/tabs/GawaTab.ts`
- `src/tabs/SynthesisTab.ts`
- `src/views/LinkModal.ts`
- `src/modals/ConvertToTaskModal.ts`
- `src/modals/EditThoughtModal.ts`
- `src/modals/EditTaskModal.ts`